

RAN-Aware Delay Compensation for Delay-Sensitive Protocols in Cellular Networks

Yuxin Liu*
University at Buffalo SUNY
Buffalo, USA
yuxinliu@buffalo.edu

Tianyang Zhang*
University at Buffalo SUNY
Buffalo, USA
tzhang52@buffalo.edu

Qiang Wu
University at Buffalo SUNY
Buffalo, USA
qwu26@buffalo.edu

Ju Ren
Tsinghua University
Beijing, China
renju@tsinghua.edu.cn

Kyle Jamieson
Princeton University
Princeton, USA
kylej@princeton.edu

Yaxiong Xie†
University at Buffalo SUNY
Buffalo, USA
yaxiong@buffalo.edu

Abstract

Delay-based protocols rely on end-to-end delay measurements to detect network congestion. However, in cellular networks, Radio Access Network (RAN) buffers introduce significant delays unrelated to congestion, fundamentally challenging these protocols' assumptions. We identify two major types of RAN buffers - retransmission buffers and uplink scheduling buffers - that can introduce delays comparable to congestion-induced delays, severely degrading protocol performance. We present CellNinja, a software-based system providing real-time visibility into RAN operations, and Gandalf, which leverages this visibility to systematically handle RAN-induced delays. Unlike existing approaches that treat these delays as random noise, Gandalf identifies specific RAN operations and compensates for their effects. Our evaluation in commercial 4G LTE and 5G networks shows that Gandalf enables substantial performance improvements - up to 7.49× for Copa and 9.53× for PCC Vivace - without modifying the protocols' core algorithms, demonstrating that delay-based protocols can realize their full potential in cellular networks.

CCS Concepts

• **Networks** → **Network protocol design**; **Cross-layer protocols**; **Network mobility**.

Keywords

Radio Access Network, Congestion Control, 5G, LTE

ACM Reference Format:

Yuxin Liu, Tianyang Zhang, Qiang Wu, Ju Ren, Kyle Jamieson, and Yaxiong Xie. 2026. RAN-Aware Delay Compensation for Delay-Sensitive Protocols in Cellular Networks. In *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys '26)*, May 11–14, 2026, Saint Malo, France. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3774906.3802791>

*Co-primary authors.

†Yaxiong Xie is the corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

SenSys '26, Saint Malo, France

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2309-4/26/05

<https://doi.org/10.1145/3774906.3802791>

1 Introduction

Efficient congestion control is paramount for application performance, and timely detection of congestion is crucial for minimizing latency. End-to-end delay serves as a critical early indicator: as congestion builds, packets experience increased buffering, leading to delay increases that precede packet loss. Consequently, protocols leveraging end-to-end delay can react proactively, maximizing network utilization while minimizing buffering and latency. This principle underpins numerous transport protocols and real-time applications, from classic TCP variants like TCP Vegas [10, 26, 37] to modern video conferencing systems [12, 25]. Even machine learning-based congestion control algorithms [13, 14, 32, 38, 39, 42, 46], integrate delay signals into their optimization objectives, recognizing delay's value as an early congestion indicator.

For delay to serve as an accurate and timely indicator of network congestion, delay variations should mainly reflect packet queuing at bottleneck links. The principle is straightforward: when network resources become scarce, packets queue at bottleneck links, causing measurable increases in end-to-end delay. By monitoring these delay increases, protocols can detect emerging congestion before packet loss occurs, enabling them to proactively adjust their sending rates. This relationship between queuing and congestion forms a fundamental assumption for modern delay-based protocols—that increased delays directly signal network resource exhaustion.

However, our investigation reveals a key challenge in cellular networks: the RAN contains multiple *internal buffers* where packets are often queued for reasons unrelated to congestion. We identify two major RAN buffers that significantly impact end-to-end delay: **1) Retransmission Buffer (§3.1)**. To ensure reliable transmission over error-prone wireless links, the cellular network implements retransmissions at two protocol layers. When transmission errors occur, subsequent packets wait in retransmission buffers until recovery succeeds. The first layer enables quick recovery with relatively short delays (around 8ms), while the second layer provides additional reliability with longer delays (60-120ms). This two-layer approach and its buffering are fundamental to reliable wireless communication, operating independently of application traffic patterns. **2) Uplink Scheduling Buffer (§3.2)**. Before any device can upload data, it must receive permission from the base station. This coordination requires devices to regularly report their buffer status to the base station, which then schedules transmission opportunities. During this process, packets wait in an uplink buffer between status

reporting and receiving permission to transmit. This scheduling mechanism is essential for managing the shared wireless medium among multiple devices, and its operation is determined by cellular protocols rather than application behavior.

These buffering mechanisms are not optional features—they are fundamental to how cellular networks operate, ensuring reliable communication and efficient resource sharing among multiple devices. However, they introduce substantial delay variations that can reach tens of milliseconds, comparable to or even exceeding typical congestion-induced delays. Because these delays arise from essential cellular operations rather than network congestion, they fundamentally challenge the assumptions of delay-based protocols. Such protocols are intentionally sensitive to delay increases for low-latency operation, but this makes them vulnerable to RAN-induced delays that mimic congestion buffering—an issue that cannot be addressed by simply tuning the algorithms to be less “nervous.”

Today’s delay-sensitive protocols lack visibility into these RAN internal operations, leaving them unable to distinguish congestion-induced delays from RAN buffering delays. This visibility gap exists because cellular networks are largely closed systems—chipset manufacturers and network operators keep their implementations proprietary, and RAN operations[17] are abstracted away from higher protocol layers. Without access to RAN operational information, protocols can only treat these RAN-induced delays as random noise [7, 28, 36, 47], applying filtering techniques in an attempt to extract true congestion signals. However, our analysis reveals that this approach fundamentally falls short. The delay patterns introduced by RAN buffers are neither random nor noise—they are systematic effects tied to specific RAN operations. Simply filtering these delays fails to address their root cause and leaves protocols struggling to distinguish between actual congestion and RAN effects.

To close this visibility gap, we build CellNinja, a software system that provides *comprehensive, real-time visibility into 5G Standalone (SA) RAN operations directly on commodity smartphones*. At its core, CellNinja continuously collects and decodes rich modem diagnostics at TTI granularity. It then propagates these signals to higher-layer logic through a *novel low-latency diagnostic pipeline* (§4.2.1), delivering them almost instantaneously as RAN events occur. This real-time feedback loop is critical, as it establishes a *high-fidelity, closed-loop control system* that enables higher-layer applications to perceive and react to RAN dynamics as they happen. Such immediacy allows precise, timely actions based on the instantaneous state of the wireless link, a capability previously unattainable.

CellNinja marks a fundamental advance over prior art. Passive sniffers like NG-Scope [40] and NR-Scope [34] require external hardware and offer only partial visibility into encrypted user sessions, making them unsuitable for on-device adaptation. Conversely, existing on-device diagnostic tools like MobileInsight [22] and QC-Super [2] are designed for offline analysis; they lack support for 5G SA networks [8, 15, 21, 23, 43, 44] and, most importantly, the real-time feedback loop necessary for closed-loop control. By overcoming these limitations, CellNinja provides the foundational visibility needed to build a new class of cellular-aware applications.

Building on CellNinja’s visibility into RAN operations, we develop Gandalf, a system that fundamentally changes how protocols handle RAN-induced delays. Instead of attempting to filter out

Network Type	COPA thput		COPA delay		PCC thput		PCC delay	
	DL	UL	DL	UL	DL	UL	DL	UL
5G	7.5×	2.8×	1.1×	1.0×	9.5×	2.2×	1.0×	1.0×
LTE	6.3×	3.8×	1.1×	1.0×	5.3×	2.9×	1.0×	1.0×

Table 1: Performance gains with Gandalf integration: throughput (thput) and delay ratios for COPA and PCC in 5G SA and LTE.

these delays as noise, Gandalf enables protocols to understand and systematically account for them. Using the real-time RAN information from CellNinja, Gandalf identifies specific RAN operations and compensates for their delay effects (§4.3). For retransmission buffers, Gandalf leverages a key insight: retransmissions are discrete, independent events with well-defined temporal boundaries. By tracking the start and end of each retransmission event through RAN messages, Gandalf can precisely identify affected packets and subtract the corresponding delay inflation (§4.3.1). For uplink scheduling buffers, Gandalf exploits its understanding of the buffer status reporting mechanism to perform informed filtering—rather than applying generic smoothing techniques, it uses the actual reporting interval from RAN to determine the precise frequency of delay variations these buffers introduce (§4.3.2). This RAN-aware filtering ensures we remove only the delay components that match known RAN behaviors while preserving true congestion signals. This systematic approach allows delay-sensitive protocols to operate in cellular networks as effectively as they do in traditional networks, maintaining their core congestion control logic while gaining awareness of RAN-specific behaviors.

Our extensive evaluation reveals that RAN-induced delays severely degrade the performance of three representative delay-sensitive protocols. COPA’s [7] rate control becomes excessively conservative, as it misinterprets normal RAN buffering as severe network congestion. PCC Vivace [14] experiences erratic rate adjustments due to RAN-induced delay variations, leading to significant throughput degradation. WebRTC’s video quality suffers from frequent, unnecessary quality reductions when its congestion control mechanism mistakes RAN delays for network congestion. By integrating Gandalf with these protocols, we achieve dramatic performance improvements without modifying their core algorithms, as shown in Table 1. In commercial 4G LTE and 5G networks, Gandalf improves throughput by up to 7.5× for COPA and 9.5× for PCC while maintaining similar latency. These results demonstrate that by providing protocols with the ability to distinguish between RAN-induced delays and true congestion, Gandalf enables existing delay-based algorithms to realize their full potential in cellular networks.

Our paper makes the following contributions:

- We identify and quantify two essential sources of non-congestion delay in cellular RAN. We show that the resulting delay variations, long regarded as random cellular noise, are actually systematic effects rooted in essential RAN operations, and we provide a complete explanation of the underlying mechanisms that distort end-to-end delay signals.
- We design and implement CellNinja, the first software-only system that provides real-time, fine-grained visibility into 5G SA RAN operations directly on commodity mobile devices.

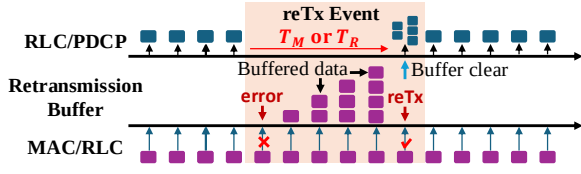


Figure 1: Retransmission buffer: packets received between the original transmission and retransmission are buffered and released together upon successful retransmission.

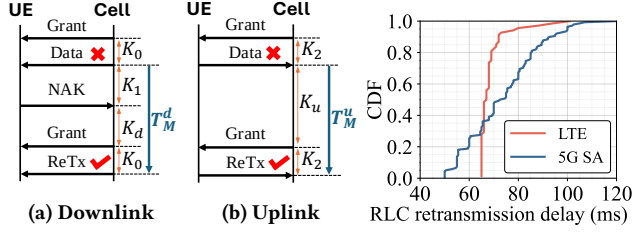


Figure 2: Time sequence of MAC layer retransmission operations: RLC retransmission delay (a) downlink and (b) uplink.

- We develop Gandalf, a cross-layer mechanism that leverages this visibility to systematically compensate for RAN-induced delays by restoring correct delay signals before they reach congestion control. Our approach explicitly isolates and removes structural delay components, without modifying the core logic of existing congestion control algorithms.
- Through extensive experiments in commercial 4G/5G networks, we demonstrate that RAN-induced delays significantly mislead delay-based congestion control in real-world deployments, and our approach yields substantial throughput gains—up to 9.53×—for widely-used protocols like COPA and PCC Vivace.

2 Primer of Cellular Networks

Frame Structure. The system frame (SFN) lasts 10ms in both LTE and 5G networks. LTE [4] divides each SFN into 10 subframes of 1ms, while 5G uses variable-duration slots determined by the subcarrier spacing configuration [3, 5].

Transmission Time Interval. A *transmission time interval* (TTI) defines the minimum time unit during which a cellular base station can schedule radio resources for user uplink or downlink transmissions. In LTE, a TTI equals one subframe (1ms), while in 5G SA, a TTI corresponds to one slot.

PDCP, RLC, MAC, and PHY Layer Interactions. The packet data convergence protocol (PDCP) layer passes its protocol data units (PDUs) to the radio link control (RLC) layer, which forwards them to the medium access control (MAC) layer. The MAC layer adaptively processes RLC PDUs based on available PHY radio resources: it may segment one RLC PDU into multiple MAC PDUs, or combine multiple RLC PDUs into a single MAC PDU. Each MAC PDU is then transmitted through the physical (PHY) layer within one TTI.

Two-level Retransmission for Reliability. Cellular networks implement a two-level retransmission mechanism to maximize data reliability. When a MAC PDU is received with errors, the MAC layer attempts retransmission to recover the lost data. If MAC layer retransmissions fail to recover the data, resulting in a corrupted

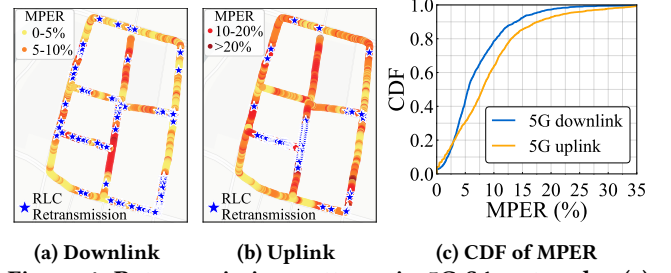


Figure 4: Retransmission patterns in 5G SA networks: (a) downlink and (b) uplink MPER across locations, with stars indicating RLC retransmission events. Colors represent different MPER ranges. (c) CDF of per-second MPER.

or partially received RLC PDU, the RLC layer initiates its own retransmission process. Both RLC and MAC layers employ *automatic repeat request* (ARQ) schemes for retransmission: the receiver signals successful reception with an acknowledgment (ACK) or failure with a negative acknowledgment (NACK), prompting the sender to retransmit the corresponding data as needed.

3 Internal Buffers inside RAN

This section dissects the root causes of RAN-induced delay by providing a detailed analysis of the internal buffers responsible. Our investigation identifies two dominant sources (§3.1 and §3.2) of delay variation that are inherent to cellular network design yet invisible to end-to-end protocols.

3.1 Retransmission Buffer

In this section, we examine cellular networks' retransmission mechanisms, specifically focusing on retransmission buffer management and their effects on packet transmission.

3.1.1 Retransmission Buffer and its Impact.

In-order Delivery and Retransmission Buffer. The ARQ scheme introduces a delay between the original transmission and retransmission, as illustrated in Figure 1. We denote these retransmission delays as T_M and T_R for MAC and RLC layers respectively. During the retransmission period, data transmission continues uninterrupted. To maintain in-order delivery, all data received during this interval is stored in a *retransmission buffer*, which resides at the UE for downlink traffic and at the base station for uplink traffic, as shown in Figure 1. Upon successful retransmission, the entire contents of the buffer are delivered simultaneously to the next protocol layer: to the RLC layer for MAC layer retransmissions, and to the PDCP layer for RLC layer retransmissions.

Retransmission Event. We define a *retransmission event* (or *reTx event*) as the process of recovering from a single data error. Each retransmission event spans a duration of T_M or T_R for MAC and RLC retransmissions respectively, as shown in Figure 1. The retransmission buffer remains non-empty only during these events.

Impact on Packet Delivery and Delay. During each reTx event, all incoming packets are held in the retransmission buffer, completely suspending all packet delivery to higher protocol layers and creating an *idle period* equal to the retransmission delay duration, as shown in Figure 1. This buffering mechanism introduces additional packet delay to all packets currently stored in the buffer.

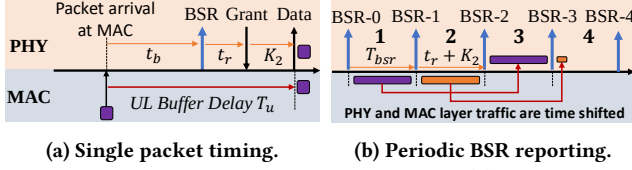


Figure 5: MAC-to-PHY packet progression: (a) decomposing MAC-to-PHY buffer delay for a packet, (b) time shift between PHY and MAC layer traffic due to periodic BSR reporting.

For packets experiencing transmission errors, this delay reaches a maximum of T_M for MAC layer traffic and T_R for RLC layer traffic.

3.1.2 Quantifying Retransmission Delays.

MAC Layer. The MAC layer retransmission follows a predefined procedure defined by the 3GPP standard, making T_M deterministic. For downlink transmission, the base station sends a grant to UE, followed by the actual data transmission, with these events separated by K_0 TTIs, as in Figure 2a. Upon detecting an error, the UE sends a NAK after K_1 TTIs, and the base station issues a new grant after K_d TTIs. Therefore, the downlink retransmission delay T_M^d is:

$$T_M^d = K_0 + K_1 + K_d \quad (1)$$

For uplink transmission, as shown in Figure 2b, the UE must receive a grant before transmitting data, with an interval of K_2 TTIs between grant and data. When an error occurs, the base station schedules another grant after K_u TTIs, resulting in an uplink retransmission delay T_M^u of:

$$T_M^u = K_2 + K_u. \quad (2)$$

The values of T_M^d and T_M^u differ between LTE and 5G networks. In LTE, both delays are fixed at 8 ms. In 5G SA networks, however, both T_M^d and T_M^u vary with the slot index i ($i \in [0, 19]$) within a system frame. We detail the derivation of these delays for both LTE and 5G in §4.4.2 using CellNinja.

RLC Layer. RLC retransmission relies on a *polling* mechanism: the sender periodically sets a polling bit in RLC PDUs to request ACK/NAK feedback and initiates retransmission upon receiving NAK. Polling frequency depends on two configurable thresholds: after every *pollPDU* PDUs or *pollByte* bytes transmitted. The timing of polling events varies because PDU transmission pace depends on MAC layer scheduling opportunities—MAC may combine multiple RLC PDUs into one TTI or segment a single RLC PDU across multiple TTIs based on available radio resources. As a result, both polling bit delivery and subsequent retransmissions experience variable delays due to this MAC layer dependency. This is evident in our measurements in Figure 3, where RLC retransmission delays T_R range from 60–100ms in LTE and 60–120ms in 5G SA networks.

3.1.3 Characterization of Retransmission Frequency.

To understand retransmission patterns in real-world scenarios, we conducted experiments with a mobile user carrying a Xiaomi 10 Lite while walking along city streets. We measured both MAC PDU Error Rate (MPER) and RLC retransmission events during continuous data transfer over 5G SA network.

Figure 4 presents our findings. Figure 4a and 4b show downlink and uplink MPER distributions across locations, with RLC retransmission events marked as stars. Figure 4c shows the CDF of per-second MPER. Our measurements reveal that MAC layer retransmissions are pervasive, with median MPER of 5.54% (downlink)

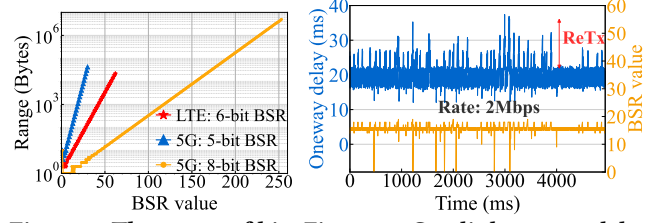


Figure 6: The range of bits BSR represents in LTE/5G. and corresponding BSR value.

and 8.00% (uplink), indicating that a substantial portion of MAC PDUs require retransmission. While RLC layer retransmissions occur less frequently, the scattered stars across uplink and downlink show they remain a consistent feature of cellular transmission. Importantly, these retransmissions are inherent RAN behaviors caused by wireless channel errors and independent of traffic patterns.

3.2 Uplink Scheduling and Uplink Buffer

Uplink Scheduling. The MAC layer of the UE cellular modem cannot transmit packets directly to the PHY layer because uplink transmissions are controlled by the base station through grants. This creates a fundamental challenge: the base station is unaware of packet arrivals at the UE and cannot proactively schedule resources. To bridge this gap, the UE sends a *buffer status report* (BSR) to inform the base station of its pending upload data volume. Upon receiving the BSR, the base station allocates appropriate resources and issues a grant to the UE, as illustrated in Figure 5a. Following grant reception, the UE must wait for K_2 TTIs before initiating the uplink transmission (§3.1). Once transmission begins, the UE periodically reports BSR with an interval of T_{bsr} , enabling the base station to track ongoing buffer status.

3.2.1 Uplink Buffer Delay.

Our analysis shows that each packet experiences a delay T_u in the UE's MAC layer *uplink buffer* before transmission to the PHY layer, as shown in Figure 5a. The total delay consists of three components:

$$T_u = t_b + t_r + K_2 \quad (3)$$

In Equation 3, the value t_b represents the interval between packet arrival and BSR reporting. Due to the periodic nature of BSR transmission, $t_b \in [0, T_{bsr}]$. The parameter K_2 is a constant system parameter. The delay t_r between BSR reception and grant transmission is determined by the base station's uplink scheduling algorithm. Our empirical measurements across four operators (two in the US and two in China) demonstrate that in both LTE and 5G SA networks, t_r is consistently configured to maintain:

$$T_{bsr} \leq t_r + K_2 \leq 2T_{bsr} \quad (4)$$

This scheduling configuration ensures that traffic arriving during the i -th BSR interval is transmitted during the $(i+2)$ -th interval, as shown in Figure 5b. Consequently, MAC layer and PHY layer traffic patterns exhibit a time shift of two BSR intervals. When sufficient uplink resources are available, the base station typically schedules all uplink traffic, including both data and BSR, within a single TTI. Figure 5b illustrates this behavior, where data accumulated during the second BSR interval is transmitted together with the third BSR. This combined transmission of BSR and user data reduces signaling

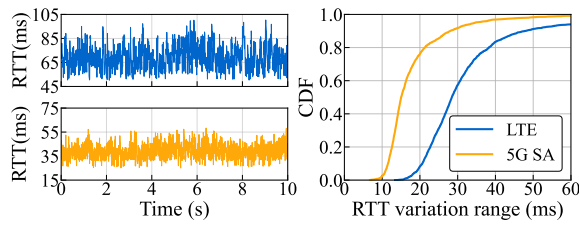


Figure 8: Collective impact of RAN-induced delay on end-to-end packet delivery.

overhead and simplifies the uplink process, as BSR transmission also requires grants from the base station.

3.2.2 Practical Buffer Estimation Challenges.

Theoretical analysis in §3.2.1 suggests that during continuous transmission, the uplink buffer should never be empty, with all packets experiencing a minimum buffering delay of one BSR interval. This fixed delay could be considered equivalent to propagation delay, as it affects only absolute delay values rather than variance. However, practical observations reveal packets occasionally experience less than one BSR interval of buffering, or even no buffering delay.

This discrepancy stems from the base station’s inability to accurately estimate the UE’s buffered data volume, due to two fundamental challenges. First, the BSR format has limited precision: rather than conveying exact buffer sizes, each BSR value represents a range that increases exponentially, as shown in Figure 6 for LTE and 5G SA networks. Second, data may be reported multiple times in successive BSRs. For instance in Figure 5b, the second BSR reports both newly accumulated data from its interval and previously reported but untransmitted data from the first interval. If the base station were to allocate resources solely based on BSR reports, it might redundantly allocate resources for the same data multiple times. To prevent this, the base station must estimate actual buffer status by correlating scheduled resources, transmitted data, and BSR reports to distinguish new data from previously reported content.

These two challenges make the base station’s estimation of buffered data imprecise and often lead to resource over-allocation. The UE utilizes any excess granted resources to transmit additional packets, effectively allowing some packets to bypass the expected buffering delay. This behavior is evident in Figure 7, which depicts the relationship between oneway delay and BSR value. With the UE strictly pacing at 2Mbps, the BSR value should remain stable. However, we observe sporadic drops of BSR to zero, indicating that the base station’s over-allocation allows the UE to transmit more data than its steady arrival rate. Note that if the UE did not strictly pace its sending rate at 2Mbps, we would observe more frequent BSR drops since varying transmission rates make it more challenging for the base station to estimate buffer status accurately. **Takeaway.** Uplink packets experience variable uplink buffering delay T_U ranging from zero to two BSR intervals.

3.3 Collective Impact of RAN Delays

The RAN introduces multiple sources of delay through MAC retransmissions, RLC retransmissions, and uplink buffering, which can affect packets either individually or in combination. The impact becomes particularly significant for RTT-based applications, as RTT

measurements encompass both paths: downlink packets may experience both MAC and RLC retransmissions and thus get delayed, while uplink packets potentially face all three delay sources.

To quantify this impact on RTT, we conducted extensive experiments between an AWS server and a UE. The server transmitted packets at 2Mbps with random inter-packet intervals, while the UE responded with 40-byte ACKs for RTT measurement. We maintained this low data rate to minimize congestion effects. The experiment comprised 400 30-second trials across one week. Figure 8a and Figure 8b show representative RTT traces for LTE and 5G networks respectively, while Figure 8c presents the CDF of RTT variation range (95th percentile minus 5th percentile RTT) across all traces. Even without network congestion, we observe significant RTT variations: individual traces show RTT variation ranges of 30.5ms in LTE and 19.1ms in 5G. These variations, stemming from RAN mechanisms rather than network congestion, can mislead applications about network conditions.

4 Mitigating Impact of Buffers inside RAN

In this section, we present our solution for mitigating RAN buffer impacts on end-to-end protocols and applications.

4.1 Overview of Solution

RAN buffer delays stem from systematic internal processes rather than random network jitter. These delays arise from protocol-driven RAN mechanisms, and their occurrence and duration are determined by internal operations such as retransmission and BSR-based scheduling. Because such behaviors are not observable from end-to-end measurements alone, their effects cannot be reliably separated from genuine congestion by transport-layer filtering. Hence, overcoming these delays requires two capabilities: (1) real-time visibility into RAN events, and (2) the ability to correlate them with data-plane traffic. The need for these capabilities motivates a cross-layer approach that combines real-time RAN visibility with data-plane delay measurements. Our key insight is to leverage the cellular modem’s dual-subsystem design: a *diagnostic subsystem* that collects internal RAN status via diagnostic (DIAG) messages, and a *data subsystem* that carries user data through the standard cellular protocol stack. These subsystems normally operate independently, each providing different information (RAN status vs. user traffic). By bridging them, we create a real-time cross-layer control path that feeds RAN insights into the data plane.

As shown in Figure 9, we realize this integration with two software-only components on the device. CellNinja attaches to the diagnostic subsystem, issuing diag commands and processing those messages to capture RAN events. Gandalf ingests CellNinja’s output and correlates these RAN insights with data-plane signals from the data subsystem (e.g., packet delays) to systematically eliminate RAN-induced delays. CellNinja provides RAN visibility, while Gandalf uses it to perform delay compensation; together, they nullify the impact of RAN buffers.

Overall, this integration yields a new framework for RAN-aware, cross-layer control. This architecture fundamentally changes how protocols interact with cellular networks by exposing RAN internals to software in real time. By establishing a tight, real-time feedback loop between RAN dynamics and transport-layer logic, our software-only approach transforms the traditionally passive

Tools	Real-Time	5G SA	NO. of LTE msg	NO. of 5G msg	Mobile-Ready	Granularity
NG-Scope [40]	✓	✗	1	0	✗ Sniffing with USRP	TTI
NR-Scope [34]	✓	✓	0	1	✗ Sniffing with USRP	TTI
MobileInsight [22]	✗	✗	57	8	✓	TTI
CellNinja	✓	✓	99	60	✓	TTI

Table 2: Comparison of cellular network monitoring tools: CellNinja enables real-time monitoring of both LTE and 5G SA networks with comprehensive message coverage, while maintaining direct mobile device deployment.

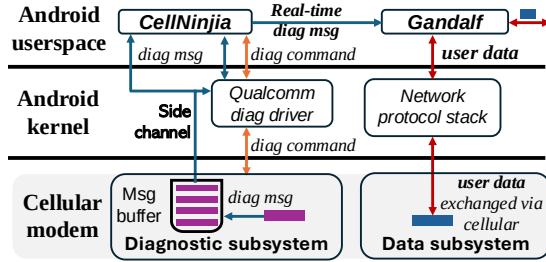


Figure 9: System architecture: CellNinja extracts real-time RAN events from the diagnostic subsystem, and Gandalf correlates them with user data from the data subsystem to compensate RAN-induced delays.

diagnostic port into an active source of control-plane information. Moreover, because CellNinja and Gandalf are pure software (no hardware modifications required), this capability is deployable on commodity devices. This, in turn, enables a broad class of optimizations previously impossible without such visibility, potentially revolutionizing how mobile networks are optimized.

4.2 CellNinja: Visibility into the RAN

Design Goals. To enable real-time optimization of cellular network performance, we need comprehensive and instantaneous visibility into RAN operations. Our system must meet the following requirements: 1) *Software-based Implementation.* The system must be directly implementable on mobile devices without requiring additional hardware or external tools. 2) *Comprehensive Parameter Access.* The tool must provide access to a wide range of internal RAN parameters shared between base stations and UEs. 3) *Fine-grained Monitoring.* The system must monitor data transmission of the cellular modem at TTI-level granularity to accurately track fine-grained RAN activities, particularly retransmission events. 4) *Real-time Operation.* The system must provide instantaneous visibility into RAN behaviors directly on mobile devices, enabling real-time correlation with data transmission and supporting immediate protocol adaptation without external processing or additional hardware. 5) *Multi-generation Support.* The system must support both LTE and 5G SA networks.

Qualcomm Diagnostic Subsystem. Qualcomm provides access to RAN internal states through a *diagnostic subsystem* (diag), whose architecture is shown in Figure 9. The cellular modem collects RAN status and statistics, encodes them into *diag messages*, and stores them in a *diag buffer*. Upon saturation, the modem aggregates all buffered messages into a single *diag message group*, transfers it to the Android kernel, and clears the buffer. A Qualcomm-implemented diag driver receives these message groups and buffers them in the kernel. The buffered message groups are reported to userspace applications upon query; otherwise, they are discarded.

Challenge. While the diag system provides valuable access to RAN status, it presents two main challenges. First, the modem’s buffering mechanism introduces substantial delay in diag message collection. Second, while Qualcomm’s diag driver is open-source, the modem firmware and its interaction protocols remain closed-source, making both communication with the modem and interpretation of its diagnostic messages challenging. This limited transparency particularly impacts the research community, requiring significant reverse engineering effort to achieve reliable system operation.

4.2.1 Design of CellNinja.

Buffer Draining. Through reverse engineering, we found that Qualcomm modems support an internal buffer draining capability disabled by default. It must be explicitly triggered via a diag command. Once activated, it forces the modem to immediately flush each diagnostic message from its internal buffer as soon as it’s generated, rather than buffering multiple messages and batching them for reporting. Enabling this hidden capability thus allows CellNinja to achieve truly instantaneous collection of RAN messages.

Experimental Verification. To verify the effect of buffer draining, we saturated the uplink to ensure continuous message generation, collected diagnostic reports in userspace, and measured the number of messages per report group. Figure 10 shows the resulting CDF. With draining enabled, nearly all groups contain exactly one message, indicating per-message reporting. Without draining, the modem buffer large groups by default, about 70 messages in LTE and 120 in 5G SA, before releasing them, causing significant delay.

To our knowledge, no open-source tool (e.g., MobileInsight [22], QCSuper [2], SCAT [1]) or commercial tools such as XCAL [6] implement this draining mechanism. CellNinja is thus the first to enable per-message reporting for real-time RAN visibility.

Side Channel. The Qualcomm diag driver handles multiple functionalities with a complex processing chain for each message received from the modem, for example, modem handles both diag messages and control messages. This extensive processing introduces significant delay between a message’s arrival at the Android kernel and its delivery to userspace. To minimize this processing latency, we implement a kernel-level side channel that immediately forwards diag messages to CellNinja upon their arrival in the Android kernel. This side channel also provides an interface for other kernel modules to access diag messages directly.

Evaluation. To evaluate our optimization, we measured the end-to-end delay from message arrival in kernel to userspace delivery, comparing our side channel against the standard diag driver path. As shown in Figure 12, side channel reduces delay by approximately 50%, with significant improvements under high data rates.

Comparison with Existing Tools. CellNinja provides comprehensive visibility into cellular RAN operations through real-time and extensive message monitoring capabilities. As shown in Table 2,

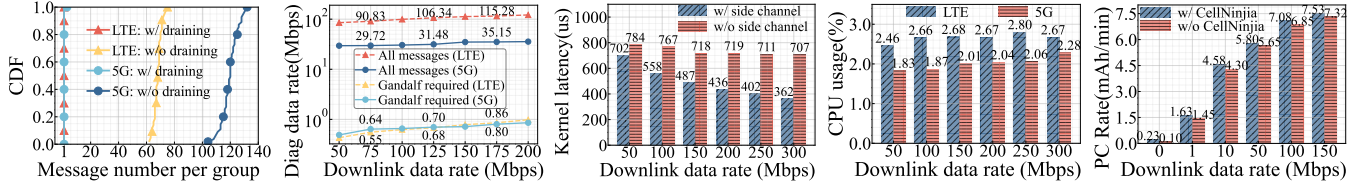


Figure 10: CDF of mes- **Figure 11: Diag data rate** **Figure 12: The median** **Figure 13: CellNinja** **Figure 14: Power con-**
sage number per group of all messages and Gan- **kernel latency with and** **CPU usage with Gandalf** **sumption (PC) rate with**
with and without drain. **dalf required messages.** **without side channel.** **required messages only.** **and without CellNinja.**

CellNinja outperforms existing cellular network monitoring tools, each evaluation metric is explained:

- **Real-Time:** Whether the tool provides instantaneous message reporting with minimal delay. CellNinja implements buffer draining mechanisms to achieve true real-time reporting.
- **5G SA:** Support for standalone 5G networks. CellNinja has been tested across multiple commercial 5G SA networks, while other tools either lack 5G SA support or have limited validation.
- **NO. of LTE/5G Msg:** Number of unique message types decoded by each tool. The LTE and 5G SA versions of CellNinja decode 99 and 60 message types respectively. In comparison, MobileInsight [22] decodes only 57 LTE message types and eight 5G message types. NG-Scope and NR-Scope focus only on control-channel DCI messages, thus supporting a single message type.
- **Mobile-Ready:** Whether the tool can run directly on mobile devices. NG-Scope and NR-Scope require external USRP (Universal Software Radio Peripheral) hardware for sniffing cellular signals.
- **Granularity:** The temporal resolution of network monitoring. All tools achieve TTI (Transmission Time Interval) level granularity.

This comparison highlights CellNinja's advantages: comprehensive message coverage, true real-time operation through buffer draining, validated 5G SA support, and direct mobile device deployment without additional hardware requirements.

4.2.2 Implementation of CellNinja.

Qualcomm chipsets map the modem diagnostic interface to the /dev/diag virtual device in OS. Consistent with existing diagnostic tools, CellNinja issues diag commands via ioctl to selectively enable/disable cellular message types and receive binary streams. As discussed in Section 4.2.1, real-time data collection requires addressing two latency sources: modem buffering and kernel processing delay. For modem buffering, CellNinja issues a privileged diagnostic command to enable the diag_peripheral_buffering_drain_immediate function [29] in the Qualcomm diagnostic subsystem, forcing the modem to flush diagnostic message immediately upon generation. For kernel processing delay, we implement a side channel by registering a custom Netlink protocol family and hooking the diag message entry point in OS. Incoming diag messages are forwarded directly to the CellNinja process via Netlink unicast.

4.2.3 System Overhead Analysis of CellNinja.

We quantify CellNinja's operational overhead through three critical metrics: the data rate of diagnostic messages from the diag subsystem, the CPU utilization and the power consumption rate.

Diagnostic Message Data Rate. The volume of diagnostic messages depends on the number of message types enabled in CellNinja. We evaluated two configurations: one with all supported

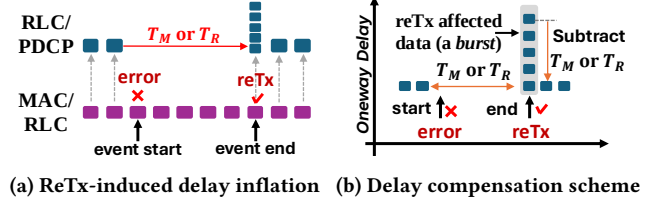


Figure 15: Retransmission delay compensation: (a) delay inflation of T_M or T_R for packets between error and retransmission, (b) subtraction of added delay from affected packets.

message types enabled, and another with only the messages required by Gandalf. We measured diagnostic message data rates while varying the download speed from 50 Mbps to 200 Mbps in both LTE and 5G SA networks. As shown in Figure 11, when all message types are enabled, LTE generates approximately 100 Mbps of diagnostic data, slightly increase corresponding to higher download rates. 5G SA produces a lower volume at around 30 Mbps. In contrast, the minimal configuration required by Gandalf generates less than 1 Mbps of diagnostic data for both LTE and 5G SA.

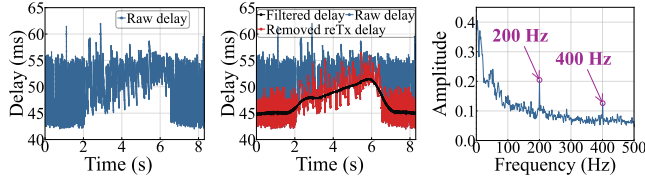
These results demonstrate two key points. First, Gandalf's minimal overhead makes it practical for continuous operation on mobile devices. Second, since Gandalf uses only a subset of CellNinja's diagnostic messages, the rich RAN information captured by CellNinja offers significant potential for other cellular network optimizations. **System Overhead.** We analyze CellNinja's runtime overhead over 24 hours under varying network conditions. Figure 13 shows that enabling CellNinja incurs only 2–3% CPU overhead, consistent across LTE and 5G SA, even under heavy downlink throughput. Figure 14 indicates that total device power consumption increases with higher traffic rates (as expected from increased radio activity), and with CellNinja active the cumulative power draw is only slightly above the baseline (no CellNinja) without exceeding a practical range. Overall, CellNinja imposes low system overhead, supporting long-term background operation on commodity smartphones.

4.3 Gandalf: Cross-layer Protocol Adaptation

This section presents Gandalf's mechanisms for protocol adaptation using CellNinja's RAN visibility. We detail our algorithms for handling retransmission delays (§4.3.1) and uplink buffering delays (§4.3.2), enabling real-time compensation for RAN-induced effects.

4.3.1 Retransmission Buffer induced Delay.

Key Observation. The retransmission buffer exhibits several key properties that enable systematic delay mitigation. First, buffer occupancy occurs exclusively during retransmission events, having no impact on packet delays during normal operation. Second,



(a) The raw delay (b) comparison (c) Spectrum after FFT
Figure 16: Processing steps to isolate network delay: (a) raw network delay measurements, (b) comparison of raw delay, retransmission-removed delay, and filtered delay, and (c) frequency spectrum after FFT showing RAN-specific components at 200Hz and 400Hz.

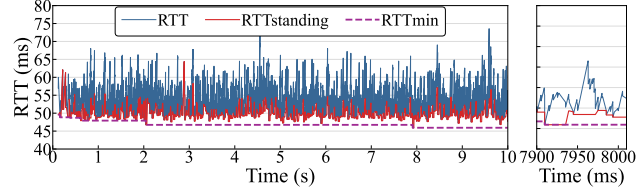


Figure 17: The workflow of COPA's queuing delay estimation. COPA estimates the queuing delay by calculating the difference between $RTT_{standing}$ and RTT_{min} .

each retransmission event occurs independently, with well-defined starting and ending points in time, just as shown in Figure 15a. Third, the impact of each event is deterministic, introducing a delay inflation of T_M for MAC layer and T_R for RLC layer retransmissions. **Delay Compensation Mechanism.** The properties of retransmission events enable a systematic approach to compensate for retransmission-induced delays. We follow a three-step process. First, we identify the temporal boundaries of each retransmission event using RAN information from CellNinja, as illustrated in Figure 15b. Second, we identify affected packets based on their characteristics: a large gap from previous packets due to the retransmission-induced idle period, followed by near-simultaneous delivery of multiple packets at the event's end. Finally, we compensate for the delay by subtracting T_M (MAC) or T_R (RLC) from the error packet and adjusting all simultaneously reported packets (a burst) to match this corrected delay. This approach effectively restores packet timing to its baseline state. The same mechanism naturally extends to multiple or consecutive retransmissions by accounting for their cumulative delay. Further details about obtaining retransmission delays (T_M and T_R) and tracking retransmission event boundaries using CellNinja are provided in §4.4.2 and §4.4.3, respectively.

Handling uplink retransmissions. For downlink traffic, correlating retransmission events with affected packets is straightforward since both RAN information and packet data are available at the UE. However, uplink presents a unique challenge: while RAN information is collected at the UE, packet delays are measured at the remote receiver. To address this challenge, we transmit retransmission event information collected using CellNinja to the remote receiver and synchronize it with packet delay measurements. The synchronization exploits the unique timing patterns of retransmissions: packets affected by MAC layer retransmission exhibit an idle period of T_M followed by a burst of packets arriving almost simultaneously, with the first packet showing a delay increase of T_M , just as shown in Figure 15a. Using these timing characteristics, we can identify

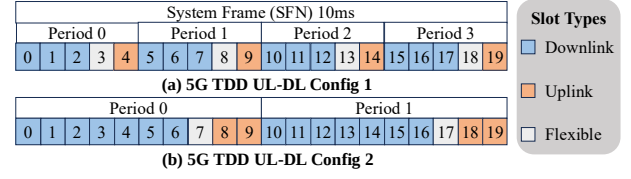


Figure 18: Two downlink and uplink configurations we measured from commercial 5G SA networks.

potential retransmission events from packet measurements alone (even though with false positives). We then search for a time shift that maximizes the overlap between packet-identified events and CellNinja-reported events. Specifically, we compute the ratio of matched events for different time shifts and select the shift that yields the highest matching ratio. This synchronized information enables accurate delay compensation even when RAN information and packet data are collected at different locations.

4.3.2 Uplink Buffer induced Delay.

Uplink buffer delay differs fundamentally from retransmission delay. While retransmissions affect packets independently and discretely, uplink buffering impacts almost every packet with varying delays. This continuous nature makes it impossible to identify and eliminate individual delay events as we did with retransmissions.

Instead, we treat the end-to-end delay (with retransmission delay removed) as a superposition of two components: uplink buffer delay and network delay (including propagation delay and network buffer queuing). By isolating these components, we can extract the true network delay signal for use by higher-layer protocols and applications. This separation is crucial because uplink buffer occupancy does not indicate network congestion and should not trigger congestion control mechanisms in end-to-end applications.

Periodic Buffer Behavior. We observe that the uplink buffer exhibits periodic filling and clearing due to the periodic nature of BSR reporting. Consequently, the uplink buffer-induced delay manifests as a periodic signal with frequency determined by the BSR interval T_{bsr} : $f_u = \frac{1}{T_{bsr}}$. The base station configures and communicates T_{bsr} to the UE, with possible values ranging from five milliseconds to several seconds. Through CellNinja, we can continuously track T_{bsr} in real-time (detailed in §4.4.1). Our measurements show that T_{bsr} is typically set to five milliseconds for both LTE and 5G.

RAN-aware Delay Filtering. We leverage this periodic behavior to isolate network delay from uplink buffer delay using signal processing techniques. After removing retransmission delays, we apply FFT to the end-to-end delay measurements, then use a low-pass filter to remove the higher-frequency buffer-induced delay component (f_u) while preserving the lower-frequency network delay variations. Network delay variations typically occur at lower frequencies as they stem from congestion buildup and dissipation, which happen over longer time scales (hundreds of milliseconds to seconds) compared to the BSR-driven buffer dynamics (typically every 5-10 milliseconds). An inverse FFT then recovers the filtered delay signal, *i.e.*, the network delay.

4.3.3 Delay Filtering in Practice.

Gandalf operates between RAN and congestion control to compensate RAN-induced delays before they affect protocol decisions. Using real-time RAN information from CellNinja, it adjusts raw

end-to-end delay by subtracting retransmission delays and removing the periodic uplink buffer component based on the BSR interval. The resulting delay reflects true congestion and can be used by congestion control algorithms without modifying their core logic.

To demonstrate how Gandalf removes RAN-induced delays in practice, Figure 16 shows the step-by-step filtering process. Figure 16a presents the raw network delay measurements. Figure 16b compares three signals: the raw delay (blue), the delay after removing retransmission effects (red), and the final filtered delay (black) representing true network congestion. The gap between raw and retransmission-removed delays reveals the impact of sporadic RAN retransmissions. Finally, Figure 16c displays the frequency spectrum of the delay signal after FFT analysis, with two distinct peaks at 200Hz and 400Hz corresponding to periodic RAN buffer behaviors. By identifying and removing these RAN-specific frequency components, Gandalf extracts the underlying network delay that accurately reflects actual network conditions.

4.4 Monitoring RAN with CellNinja

In this section, we present how to extract critical RAN parameters and monitoring RAN behaviors using CellNinja.

4.4.1 Extracting BSR Interval T_{bsr} .

There are two approaches to extracting the BSR interval. The first obtains the *periodic BSR timer* from the LTE diag messages `LTE_RRC_Connection_Setup` and `LTE_RRC_Connection_Reconfiguration`, and the 5G diag messages `NR5G_RRC_Setup` and `NR5G_RRC_Reconfiguration`. These messages configure and may be reconfigure the *periodic BSR timer*, which defines the BSR interval T_{bsr} , and any changes can be tracked by monitoring subsequent RRC reconfiguration messages. The second method computes T_{bsr} from the TTI index interval between consecutive BSR reports. The LTE diag message `LTE_ML1_UL_Transport_Block` and the 5G diag message `NR5G_L2_UL_TB` provide detailed BSR reporting information, enabling continuous tracking of T_{bsr} .

4.4.2 Extracting Retransmission Delay.

This section details how we use CellNinja's diagnostic message to extract retransmission delays for MAC T_M and RLC T_R layers.

MAC Layer Retransmission Delay T_M . In the 5G downlink, we extract retransmission timing parameters from multiple diagnostic messages. The parameter K_1 comes directly from message `NR5G_MAC_PDSCH_Status`. To derive K_0 , we first the *time resource assignment* field from `NR5G_MAC_DCI_Info`, and then map it to the *PDSCH time domain allocation list* provided by `NR5G_RRC_Reconfiguration`. The parameter K_d is computed as the time interval between the NACK and the corresponding MAC PDU. ACK/NACK feedback is obtained from the *Num HARQ ACK bits* field in `NR5G_MAC_UL_PHY_Channel_Schedule_Report`, where 0 indicates ACK and a nonzero value indicates NACK. Retransmissions are identified by checking the *redundancy version* in `NR5G_MAC_PDSCH_Status`.

In the 5G uplink, K_2 is derived by mapping the *symbol allocation index* field in `NR5G_MAC_DCI_Info` to the *PUSCH time domain allocation list* in `NR5G_RRC_Reconfiguration`. ACK/NACK feedback is inferred from *NDI* field in `NR5G_MAC_DCI_Info`,

Slot index i	0/10	1/11	2/12	3/13	5/15	6/16	7/17
K_1	4	8	7	6	4	8	7
K_d	6	6	6	6	6	6	6
T_M^d	10	14	13	12	10	14	13

(a) UL-DL configuration 1

i	0/10	1/11	2/12	3/13	4/14	5/15	6/16	7/17
K_1	8	7	7	6	5	4	12	11
K_d	5	5	5	5	5	5	5	5
T_M^d	13	12	12	11	10	9	17	16

(b) UL-DL configuration 2

Table 3: Downlink retransmission timing parameters (K_1 , K_d) and resulting delay (T_M^d) for each slot index under two TDD UL-DL configurations shown in Figure 18.

Slot index i	4/14	9/19
K_2	4	3
K_u	6	7
T_M^u	10	10

(a) UL-DL configuration 1

Slot index i	8/18	9/19
K_2	3	3
K_u	7	6
T_M^u	10	9

(b) UL-DL configuration 2

Table 4: Uplink retransmission timing parameters (K_2 , K_u) and resulting delay (T_M^u) for each slot index under two TDD UL-DL configurations shown in Figure 18.

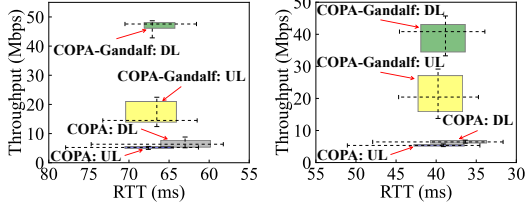
where a change indicates a successful transmission and an unchanged value indicates a NACK. The *redundancy version* field in `NR5G_MAC_UL_PHY_Channel_Schedule_Report` determines the end boundary of a retransmission event. Finally, the K_u is computed as the time interval between the NACK and the identified retransmission boundary.

RLC Layer Retransmission Delay T_R . In the 5G uplink, the *NACK SN* field in the `NR5G_RLC_DL_Status_PDU` specifies the sequence number of the PDU requiring retransmission. Using this *NACK SN*, both the initial transmission and the retransmitted PDU are traced in `NR5G_L2_UL_Data_PDU` via the *transmission type* field, which indicates whether a PDU is new or retransmitted. The T_R is computed as the time interval between the initial transmission and its corresponding retransmission.

In the LTE uplink, the *NACK SN* is extracted from `LTE_RLC_DL_AM_Control_PDU`. The corresponding *SN* field in `LTE_RLC_UL_AM_All_PDU` is then used to identify whether the given PDU is an initial transmission or a retransmission. An *SN* appearing for the first time indicates a new transmission, while a repeated *SN* signifies retransmission. The T_R is computed as the interval between the initial transmission and its retransmission.

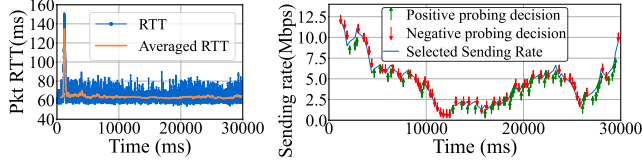
Similarly, for 5G downlink retransmission tracking, the *NACK SN* and *SN* fields are extracted from `NR5G_RLC_UL_Status_PDU` and `NR5G_L2_DL_Data_PDU`. For LTE downlink, they are obtained from `LTE_RLC_UL_AM_Control_PDU` and `LTE_RLC_DL_AM_All_PDU`. In both cases, the T_R is computed as the time interval between the initial transmission and its corresponding retransmission.

Extracted MAC Delay T_R of Commercial 5G SA. We present the extracted MAC layer retransmission delay T_M from commercial 5G SA networks under two configurations (0.5 ms slot duration)



(a) 5G SA stationary link (b) LTE stationary link (c) 5G SA under mobility (d) LTE under mobility

Figure 19: Performance comparison between COPA and COPA-Gandalf under different scenarios: static user in 5G SA (a) and LTE (b), mobile user in 5G SA (c) and LTE (d), with both uplink (UL) and downlink (DL) results shown in each subplot. Each box plot shows throughput vs. RTT, where box and edges represent 25th and 75th percentiles, and whiskers extend to 10th and 90th percentiles.



(a) The measured RTT (b) Rate probing process of PCC
Figure 21: PCC misinterprets RAN-induced RTT variations as network congestion: despite stable network conditions shown in (a), PCC makes erroneous probing decisions in (b), resulting in unnecessarily low sending rates.

in Figure 18. 5G primarily operates in TDD mode [19, 45], where uplink and downlink slot allocation is highly configurable.

Downlink Retransmission Delay. The downlink retransmission delay T_M^d is determined by three parameters: K_0 , K_1 , and K_d . Our measurements show that in 5G SA, K_0 maintains a constant value of zero, while both K_1 and K_d vary with the slot index i ($i \in [0, 19]$) within a system frame. Tables 3 provide the complete values of K_1 , K_d , and the resulting T_M^d (in number of slots) for the first and second TDD UL-DL configurations illustrated in Figure 18, respectively.

Uplink Retransmission Delay. The uplink retransmission delay T_M^u is determined by two parameters: K_2 and K_u . Our measurements show that in 5G SA, both K_2 and K_u vary with the slot index i ($i \in [0, 19]$) within a system frame. Tables 4 provide the complete values of K_2 , K_u , and the resulting T_M^u (in number of slots) for two TDD UL-DL configurations illustrated in Figure 18.

4.4.3 Tracking the Retransmission Events.

Tracking MAC layer Retransmissions. Tracking downlink MAC layer retransmission requires extracting the fields *redundancy version* and *HARQ process id* from diagnostic messages. In the LTE downlink, these fields are obtained from `LTE_ML1_PDSCH_Stat_Indication`, while in 5G they are retrieved from `NR5G_MAC_PDSCH_Status`. The *redundancy version* identifies retransmissions, and the *HARQ process id* links the retransmitted transport block to its initial transmission, defining the retransmission boundaries.

Uplink retransmissions are tracked similarly using the same key fields. However, for the uplink, these values are obtained from `LTE_ML1_PDSCH_Stat_Indication` in LTE and `NR5G_MAC_UL_PHY_Channel_Schedule_Report` or `NR5G_MAC_DCI_Info` in 5G. **Tracking RLC layer Retransmissions.** As noted in Section 4.4.2, the *NACK SN* and *SN* fields are used to trace initial transmissions and their retransmissions. In the 5G uplink, they are obtained

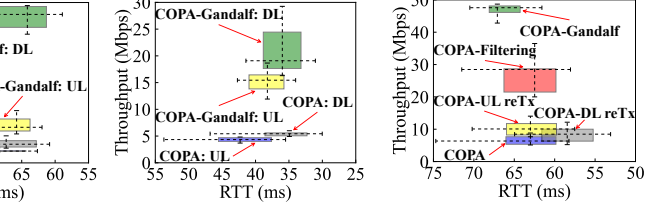


Figure 20: Ablation study of COPA variants in LTE and 5G with different Gandalf components.

from `NR5G_RLC_DL_Status_PDU` and `NR5G_L2_UL_Data_PDU`, where the *transmission type* field distinguishing new transmissions from retransmissions. LTE uplink follows the same principle using `LTE_RLC_DL_AM_Control_PDU` and `LTE_RLC_UL_AM_All_PDU`. For downlink, 5G employs `NR5G_RLC_UL_Status_PDU` and `NR5G_L2_DL_Data_PDU`, while LTE uses `LTE_RLC_UL_AM_Control_PDU` and `LTE_RLC_DL_AM_All_PDU`.

5 Gandalf for Enhanced Protocol and App

We demonstrate how Gandalf's delay compensation mechanisms improve performance of three delay-sensitive applications: Copa [7] and PCC [14] congestion control algorithms that rely on delay measurements, and WebRTC [9] real-time communication that requires accurate delay estimation.

5.1 Congestion Control: COPA

COPA [7] aims at simultaneously maximizing throughput and minimizing the delay. COPA theoretically proves that such a goal can be achieved by setting the sending rate R_s to:

$$R_s = 1/d_q \quad (5)$$

where d_q is the mean per-packet queuing delay.

Misinterpreting RAN Queuing. COPA's queuing delay measurement captures both network buffer queuing and RAN-internal buffer queuing indiscriminately, treating both of them as signals of congestion. To estimate queuing delay, COPA uses:

$$d_q = \text{RTT}_{\text{standing}} - \text{RTT}_{\text{min}} \quad (6)$$

where RTT_{min} and $\text{RTT}_{\text{standing}}$ are the minimal RTT observed within a longer (10 seconds) and shorter (one RTT) window. In cellular networks, RTT_{min} typically captures the true minimum delay during rare moments when uplink buffers are empty or near empty (BSR approaching zero), as described in §3.2.2, while $\text{RTT}_{\text{standing}}$ reflects delays with normal uplink buffer occupancy, which is typically non-empty as we demonstrate in §3.2.1. Figure 17 shows an example where COPA underutilizes the channel at 8Mbps: $\text{RTT}_{\text{standing}}$ (red line) consistently stays above RTT_{min} (purple dashed line), causing COPA to persistently estimate heavy queuing in the network, despite the absence of actual network congestion.

Misinterpreted Queuing Triggers Rate Reduction. This misinterpretation of RAN queuing directly triggers COPA's congestion response. When COPA mistakes normal RAN buffer occupancy for network congestion, it unnecessarily reduces its sending rate to clear what it perceives as harmful queues, according to Equation 5.

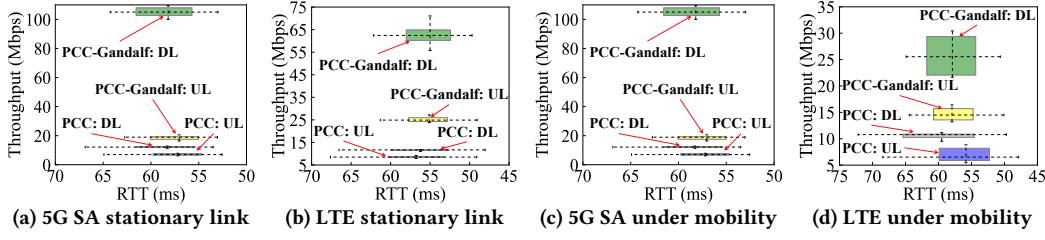


Figure 22: Performance comparison between PCC and PCC-Gandalf across different scenarios: static user in 5G SA (a) and LTE (b), mobile user in 5G SA (c) and LTE (d), with both uplink (UL) and downlink (DL) results shown in each subplot.

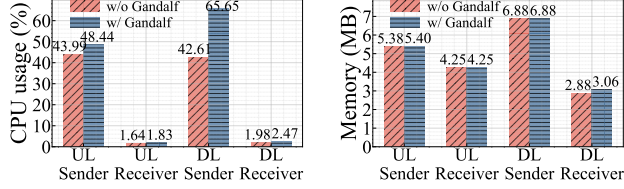


Figure 24: COPA CPU usage with/without Gandalf.

However, since RAN buffers maintains a certain occupancy level due to normal cellular operation, these rate reductions are unnecessary and counterproductive. Furthermore, since RAN buffers are persistently present by design, COPA constantly sees congestion signals and remains stuck at low data rates, severely underutilizing the available network capacity, just as evident in Figure 17.

5.1.1 Gandalf-enhanced COPA.

We use Gandalf to remove RAN-induced delays from COPA's congestion signals, allowing it to focus on true network queuing and maintain higher sending rates while remaining responsive to congestion. We denote the new system as COPA-Gandalf.

Experimental Methodology. To evaluate the performance in a realistic scenario, we adopted a common network usage configuration: a mobile client (Xiaomi 10 Lite) communicating with an AWS server [27, 30]. For each test, we established a COPA connection between the AWS server and the mobile client for 30 seconds. We repeated each test five times to account for potential network fluctuations. We also investigate both uplink and downlink performances in each test and conduct experiments inside both LTE and 5G SA networks. We test the performance with both static and mobile users. We plot the results in Figure 19.

Performance. COPA-Gandalf dramatically outperforms standard COPA across all tested scenarios while maintaining similar or lower RTT. In stationary scenarios, COPA-Gandalf improves throughput by 7.49× for downlink and 2.77× for uplink in 5G SA, and 6.34× downlink and 3.82× uplink improvement in LTE. Under mobility, COPA-Gandalf maintains throughput improvements of 7.95× downlink and 3.01× uplink in 5G SA, and 3.51× downlink and 3.56× uplink in LTE networks. These substantial improvements show that by removing RAN-induced delays, COPA can better utilize available network capacity without sacrificing latency performance.

Ablation Study. We evaluate COPA with three Gandalf variants in 5G SA—Gandalf-DL-reTx (downlink retransmission compensation only), Gandalf-UL-reTx (uplink retransmission compensation only) and Gandalf-filter (RAN-aware filtering only)—to identify which

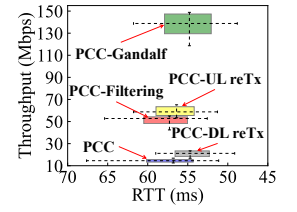


Figure 23: Ablation study comparing performance of different PCC variants under LTE and 5G.

RAN delays impact COPA. Figure 20 shows RAN-aware filtering achieves performance near full COPA-Gandalf. Since COPA's delay estimator largely reflects persistent uplink buffering, filtering this RAN-induced component improves congestion detection.

System Overhead. We evaluate Gandalf's overhead by measuring CPU and memory usage at the COPA client and server in LTE with a static user. Figure 24 shows that Gandalf brings a slight CPU increase at the receiver and a moderate increase at the sender, with the latter largely due to the 6.3× and 3.8× throughput gains in downlink and uplink. Figure 25 shows only marginal memory differences, with sender usage within 5–7 MB and receiver below 4.5 MB. Overall, Gandalf imposes limited system overhead.

5.2 Congestion Control: PCC Vivace

PCC's Rate Probing Mechanism. PCC [14] continuously probes network capacity using *monitor intervals* (MIs) that test sending rates slightly above and below the current rate. It adjusts its rate based on RTT gradients, along with throughput and loss measurements, aiming to maximize throughput while avoiding excessive queuing. Since RTT gradient is central to this process, accurate delay measurement is critical to PCC's effectiveness.

RAN-induced RTT Gradient Errors. In cellular networks, RTT measurements inherently include multiple RAN-induced delays: uplink buffering delay and retransmission delays in both uplink and downlink paths. As our previous analysis shows, these delays introduce 40–50 ms RTT fluctuations even in uncongested networks, distorting PCC's RTT gradient calculations. During higher rates probing, PCC interprets RTT gradient changes as congestion, but stem from normal RAN buffering and retransmissions. Unable to distinguish between RAN-induced variations and true congestion, PCC frequently reduces its sending rate unnecessarily, leading to suboptimal performance despite available network capacity.

Experimental Demonstration. We validate this behavior by having a UE upload data to an AWS server using PCC. Figure 21a shows RTT fluctuations caused only by the RAN-induced variations without any significant network queuing, indicating an uncongested network. However, PCC repeatedly misinterprets these RAN-induced RTT variations as congestion signals during its rate probing. As shown in Figure 21b, despite available network capacity, PCC's sending rate remains low levels (below 7.5 Mbit/s) due to these false congestion signals. Notably, while PCC employs low-pass filtering on RTT measurements it can't effectively remove RAN-induced variations as it lacks the RAN internal information needed to distinguish between RAN delays and actual network congestion.

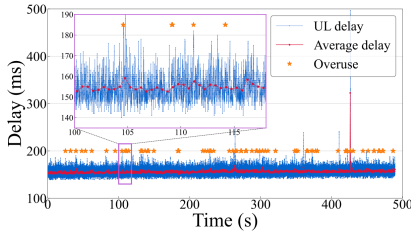


Figure 26: The original and mean uplink oneway delay measured when a UE uploads its video to an SFU server using a WebRTC connection.

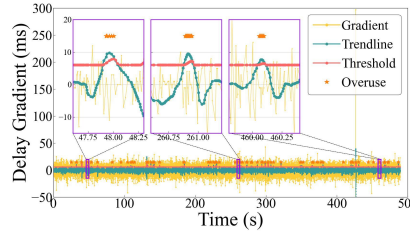


Figure 27: The process of detecting an overuse. GCC calculates the delay gradient, filters it using a Trendline filter and compares it with a threshold.

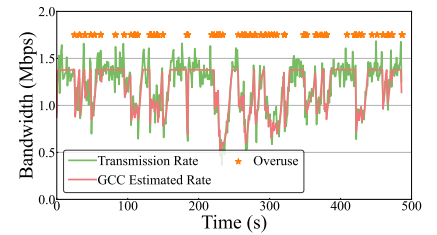
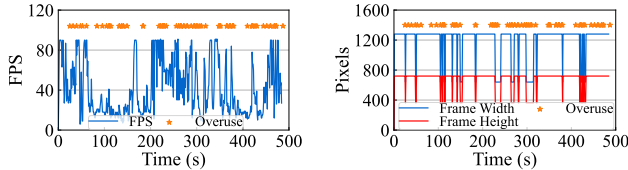


Figure 28: The rate estimated by GCC and the final video streaming rate generated by UE. Frequently detected overuse severely affects GCC's rate estimation.



(a) Frames per second (FPS)

(b) Frame resolution

Figure 29: The GCC estimated rate and video quality parameters such as FPS, Quantization parameter, and Resolution are affected by the Overuse signal.

5.2.1 Gandalf-enhanced PCC.

Building on the above analysis, we use Gandalf to remove RAN-induced variations from PCC's RTT gradient calculations. This enables PCC to estimate network capacity more accurately while remaining responsive to true congestion. We denote the resulting system as PCC-Gandalf.

Experimental Methodology. Following the same experimental setup as in §5.1.1, we evaluate PCC and PCC-Gandalf under both static and mobile scenarios in LTE and 5G SA.

Performance. PCC-Gandalf similarly achieves great throughput gains across all scenarios while maintaining comparable or lower RTT. In stationary environments, PCC-Gandalf improves throughput by 9.53× for downlink and 2.19× for uplink in 5G SA, and 5.34× downlink and 2.93× uplink improvement in LTE. Under mobility, PCC-Gandalf improves throughput of 8.65× downlink and 2.72× uplink in 5G SA, and 2.37× downlink and 2.22× uplink in LTE. These consistent improvements show that by providing accurate delay signals without RAN-induced variations, PCC can better optimize its sending rates while maintaining low latency.

Ablation Study. We evaluate PCC with three Gandalf variants in 5G SA—Gandalf-DL-reTx, Gandalf-UL-reTx, and Gandalf-filter—to identify which RAN delays impact PCC. Figure 23 shows all mechanisms improve performance except downlink retransmission compensation due to better channel quality and fewer retransmissions. Since PCC probes rates using RTT gradients, retransmission delays and uplink buffering distort its signal. Removing these RAN-induced variations improves rate estimation and adaptation.

5.3 Real-Time Communication: WebRTC

WebRTC Framework. WebRTC [9] is an open-source framework for real-time video communication. It uses Google Congestion Control (GCC) [11] to estimate available bandwidth and feeds this estimate to the video encoder, which dynamically adjusts frame rate and resolution (both width and height) to match network capacity.

Core Idea of GCC. WebRTC [9] leverages the oneway delay gradient d_m to signal channel congestion. GCC builds upon this principle but enhances robustness by applying a Trendline filter to the raw gradient, reducing the impact of random noise. It then compares the filtered delay gradient d_t to a threshold γ to guide its rate estimation A_r . Specifically, if d_t surpasses the threshold, GCC identifies the network as *overused* and adjusts its rate estimation A_r downwards. Conversely, if d_t falls below the negative threshold $-\gamma$, indicating *underuse*, GCC increases its rate estimation A_r accordingly. In cases where d_t falls within the range of $[-\gamma, \gamma]$, GCC recognizes the network as operating normally and maintains its current estimation of A_r . Moreover GCC dynamically adapts its threshold γ in response to evolving network conditions.

Uplink Delay Variance Misleads GCC. RAN induces significant uplink delay jitter, as shown in Section §3, which could mislead GCC. We stream video from a UE to an AWS SFU via WebRTC for 200 seconds and log per-packet oneway delay, calculate the average oneway delay every 500 ms and plot the data together with the overuse detected by GCC, in Figure 26. The mean delay remains stable, indicating no congestion. On the other hand, significant network jitters are introduced by RAN operation, which misleads the GCC to generate a substantial amount of overuse. To further verify that, we also plot the raw delay gradient d_m , the filtered gradient d_t , the threshold γ in Figure 27. We see that jitter produces dynamic nonzero gradients that cannot be filtered by the Trendline filter, often exceeding the threshold and triggering false overuse.

Figure 28 illustrates how GCC's estimated data rate A_r sharply decreases upon detecting overuse. While this conservative behavior is intended to prevent congestion collapse (which is positive), it can also cause excessive network underutilization in scenarios like ours, where overuse detection is triggered mostly by network jitter. The lowered capacity estimate at encoder reduces video quality. As shown in Figures 29a and 29b, the encoder adjusts the video frame rate and resolution, even without actual congestion. These unnecessary video quality fluctuations degrade user experience.

6 Related Work

Prior research has explored cellular latency in specific scenarios, including handovers during mobility [18, 28, 36], discontinuous reception (DRX) during power-saving mode [16], and Radio Resource Control (RRC) states [31]. In contrast, we examine subframe-level RAN-induced delay jitter during steady download without handovers or DRX. By focusing on this setting, our results provide insights into mitigating delay jitter in everyday cellular use.

Several recent systems have explored cross-layer approaches to improve transport performance. PBE-CC [41] and CQIC [24] expose PHY/MAC-layer capacity or scheduling signals to guide transport-layer rate control. L4Span [35] advocates explicit congestion signaling from the RAN to Layer-4 endpoints. Biscay [20] extracts modem KPIs (e.g., HARQ and MCS) to infer wireless capacity for rate adaptation. LRP [33] reduces scheduling request (SR) delay via prefetching but mainly benefits sporadic small-packet traffic. Rather than exposing PHY/MAC-layer information or adding signaling, we show that RAN retransmission and uplink scheduling introduce structured delay that distorts delay-based control. Instead of treating this jitter as noise, we identify its structural components and compensate for them before using delay for rate control.

7 Discussion and Generalisability

Generalisability Across Modems. CellNinja currently targets the Qualcomm Diagnostic Subsystem due to its widespread deployment. Evidence from existing cellular network monitoring tools [6, 22] confirms that diagnostic subsystem exists among modem vendors. While the commands and message formats are different, their functional role remain consistent. Therefore, the core design logic of CellNinja are vendor-agnostic and can be adapted to other vendors' modem only with targeted engineering efforts. Furthermore, we will open source CellNinja on GitHub (https://github.com/NEMO-LAB-repo/CellNinja_desktop and https://github.com/NEMO-LAB-repo/CellNinja_Asn1_Library) with a comprehensive roadmap for multi vendor integration.

Applicability to Congestion Control Algorithms. Gandalf restores delay signals by separating RAN-induced delay from network delay before they're used by congestion control. Therefore, it benefits to congestion control algorithms that rely on delay as a control signal, purely loss-based schemes may gain less. As modern transport increasingly incorporates delay to improve responsiveness and latency, the accuracy of delay signals are broadly relevant across congestion control architectures. In future we will integrate Gandalf with more modern algorithms and evaluate its performance.

8 Conclusion

This paper presents CellNinja and Gandalf to address the challenge of RAN-induced delays in cellular networks. By providing real-time visibility into RAN operations and systematically handling RAN-induced delays, our solution enables delay-based protocols to achieve their full potential in cellular networks without modifying their core logic. This work does not raise any ethical issues.

Acknowledgments

We sincerely appreciate the insightful feedback from the anonymous reviewers and the shepherd. This material is based upon work supported by the National Science Foundation under Award No. CNS-2223556.

References

- [1] 2025. *fgsect/scat*. <https://github.com/fgsect/scat> <https://github.com/fgsect/scat>.
- [2] 2025. *P1sec/QCSuper*. <https://github.com/P1sec/QCSuper>.
- [3] 3GPP. 2020. 3GPP TS 38.211 version 16.2.0 Release 16; 5G; NR; Physical channels and modulation.
- [4] 3GPP. 2017. Release 14 Description; Summary of Rel-14 Work Items. <http://www.3gpp.org/DynaReport/21914.htm>
- [5] 3GPP. 2022. TS 138 213 - V17.1.0 - 5G; NR; Physical layer procedures for control (3GPP TS 38.213 version 17.1.0 Release 17). (2022).
- [6] Accuver. 2025. XCAL Mobile Network Drive Test Software. <https://www.accuver.com/products/network-optimization/XCAL>
- [7] Venkat Arun and Hari Balakrishnan. 2018. Copa: Practical Delay-Based Congestion Control for the Internet. In *Proceedings of the Applied Networking Research Workshop* (Montreal QC Canada, 2018-07-16). ACM, 19–19. doi:10.1145/3232755.3232783
- [8] Arjun Balasingam. [n. d.]. Application-Level Service Assurance with 5G RAN Slicing. ([n. d.]).
- [9] Niklas Blum, Serge Lachapelle, and Harald Alvestrand. 2021. WebRTC: real-time communication for the open web platform. 64, 8 (2021), 50–54. doi:10.1145/3453182
- [10] Lawrence S. Brakmo, Sean W. O'Malley, and Larry L. Peterson. 1994. TCP Vegas: new techniques for congestion detection and avoidance. 24, 4 (1994), 24–35. doi:10.1145/190809.190317
- [11] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2016. Analysis and design of the google congestion control for web real-time communication (WebRTC). In *Proceedings of the 7th International Conference on Multimedia Systems* (New York, NY, USA, 2016-05-10) (*MMSys '16*). Association for Computing Machinery, 1–12. doi:10.1145/2910017.2910605
- [12] Sandesh Dhawaskar Sathyanarayana, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2023. Converge: QoE-driven Multipath Video Conferencing over WebRTC. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York NY USA, 2023-09-10). ACM, 637–653. doi:10.1145/3603269.3604822
- [13] Mo Dong, Qingxi Li, Doron Zarchy, Godfrey P. Brighten, and Michael Schapira. 2015. PCC: Re-architecting Congestion Control for Consistent High Performance. (2015).
- [14] Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighten Godfrey, and Michael Schapira. 2018. PCC Vivace: Online-Learning Congestion Control. (2018).
- [15] Stavros Eleftherakis, Timothy Otim, Giuseppe Santaromita, Almudena Díaz Zayas, Domenico Giustiniano, and Nicolas Kourtellis. 2024. Demystifying Privacy in 5G Stand Alone Networks. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (Washington D.C. DC USA, 2024-12-04). ACM, 1330–1345. doi:10.1145/3636534.3690696
- [16] Rostand A. K. Fezeu, Eman Ramadan, Wei Ye, Benjamin Minneci, Jack Xie, Arvind Narayanan, Ahmad Hassan, Feng Qian, Zhi-Li Zhang, Jaideep Chandrashekar, and Myungjin Lee. 2023. An In-Depth Measurement Analysis of 5G mmWave PHY Latency and Its Impact on End-to-End Delay. In *Passive and Active Measurement*, Anna Brunstrom, Marcel Flores, and Marco Fiore (Eds.). Vol. 13882. Springer Nature Switzerland, 284–312. doi:10.1007/978-3-031-28486-1_13 Series Title: Lecture Notes in Computer Science.
- [17] Xenofon Foukas, Bozidar Radunovic, Matthew Balkwill, and Zhihua Lai. 2023. Taking 5G RAN Analytics and Control to a New Level. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking* (Madrid Spain, 2023-07-10). ACM, 1–16. doi:10.1145/3570361.3592493
- [18] Ahmad Hassan, Arvind Narayanan, Anlan Zhang, Wei Ye, Ruiyang Zhu, Shuwei Jin, Jason Carpenter, Z. Morley Mao, Feng Qian, and Zhi-Li Zhang. 2022. Vivisecting mobility management in 5G cellular networks. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam Netherlands, 2022-08-22). ACM, 86–100. doi:10.1145/3544216.3544217
- [19] Rostand A. K. Fezeu, Claudio Fiandrino, Eman Ramadan, Jason Carpenter, Lilian Coelho De Freitas, Faaiq Bilal, Wei Ye, Joerg Widmer, Feng Qian, and Zhi-Li Zhang. 2024. Unveiling the 5G Mid-Band Landscape: From Network Deployment to Performance and Application QoE. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 358–372. doi:10.1145/3651890.3672269
- [20] Jon Larrea, Tanya Shreedhar, and Mahesh K Marina. 2025. BISCAY: Practical Radio KPI Driven Congestion Control for Mobile Networks. *arXiv preprint arXiv:2509.02806* (2025).
- [21] Qianru Li, Zhehui Zhang, Yanbing Liu, Zhaowei Tan, Chunyi Peng, and Songwu Lu. 2023. CA++: Enhancing Carrier Aggregation Beyond 5G. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking* (Madrid Spain, 2023-07-10). ACM, 1–14. doi:10.1145/3570361.3592500
- [22] Yuanjie Li, Chunyi Peng, Zengwen Yuan, Jiayao Li, Haotian Deng, and Tao Wang. 2016. Mobileinsight: extracting and analyzing cellular network information on smartphones. In *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking* (New York City New York, 2016-10-03). ACM, 202–215. doi:10.1145/2973750.2973751
- [23] Zhutian Liu, Qing Deng, Zhaowei Tan, Zhiyun Qian, Xinyu Zhang, Ananthram Swami, and Srikanth V. Krishnamurthy. 2024. M2HO: Mitigating the Adverse Effects of 5G Handovers on TCP. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (Washington D.C. DC USA, 2024-12-04). ACM, 1089–1103. doi:10.1145/3636534.3690680
- [24] Feng Lu, Hao Du, Ankur Jain, Geoffrey M Voelker, Alex C Snoeren, and Andreas Terzis. 2015. CQIC: Revisiting cross-layer congestion control for cellular networks.

- In *Proceedings of the 16th International Workshop on Mobile Computing Systems and Applications*. 45–50.
- [25] Zili Meng, Yaning Guo, Chen Sun, Bo Wang, Justine Sherry, Hongqiang Harry Liu, and Mingwei Xu. 2022. Achieving consistent low latency for wireless real-time communications with the shortest control loop. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam Netherlands, 2022-08-22). ACM, 193–206. doi:10.1145/3544216.3544225
 - [26] Ayush Mishra, Lakshay Rastogi, Raj Joshi, and Ben Leong. 2024. Keeping an Eye on Congestion Control in the Wild with Nebby. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 136–150. doi:10.1145/3651890.3672223
 - [27] Arvind Narayanan, Eman Ramadan, Jason Carpenter, Qingxu Liu, Yu Liu, Feng Qian, and Zhi-Li Zhang. 2020. A First Look at Commercial 5G Performance on Smartphones. In *Proceedings of The Web Conference 2020* (Taipei Taiwan, 2020-04-20). ACM, 894–905. doi:10.1145/3366423.3380169
 - [28] Yunzhe Ni, Taide Liu, Yihua Cheng, Zhiyao Ma, and Jing Wang. [n. d.]. Polycorn: Data-driven Cross-layer Multipath Networking for High-speed Railway through Composable Schedulerlets. ([n. d.]).
 - [29] Quectel. [n. d.]. Qualcomm Linux Modems by Quectel & Co. <https://osmocom.org/projects/quectel-modems/wiki/Diag>
 - [30] Muhammad Iqbal Rochman, Wei Ye, Zhi-Li Zhang, and Monisha Ghosh. 2024. A Comprehensive Real-World Evaluation of 5G Improvements over 4G in Low- and Mid-Bands. In *2024 IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN)* (Washington, DC, USA, 2024-05-13). IEEE, 257–266. doi:10.1109/DySPAN60163.2024.10632844
 - [31] Sanae Rosen, Haokun Luo, Qi Alfred Chen, Z. Morley Mao, Jie Hui, Aaron Drake, and Kevin Lau. 2014. Discovering fine-grained RRC state dynamics and performance impacts in cellular networks. In *Proceedings of the 20th annual international conference on Mobile computing and networking* (Maui Hawaii USA, 2014-09-07). ACM, 177–188. doi:10.1145/2639108.2639115
 - [32] Anirudh Sivaraman, Keith Winstein, Pratiksha Thaker, and Hari Balakrishnan. 2014. An experimental study of the learnability of congestion control. In *Proceedings of the 2014 ACM conference on SIGCOMM* (Chicago Illinois USA, 2014-08-17). ACM, 479–490. doi:10.1145/2619239.2626324
 - [33] Zhaowei Tan, Jinghao Zhao, Yuanjie Li, Yifei Xu, and Songwu Lu. 2021. Device-Based LTE Latency Reduction at the Application Layer. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)* (2021). 471–486. <https://www.usenix.org/conference/nsdi21/presentation/tan>
 - [34] Haoran Wan, Xuyang Cao, Alexander Marder, and Kyle Jamieson. 2024. NR-Scope: A Practical 5G Standalone Telemetry Tool. In *Proceedings of the 20th International Conference on emerging Networking EXperiments and Technologies* (Los Angeles CA USA, 2024-12-09). ACM, 73–80. doi:10.1145/3680121.3697808
 - [35] Haoran Wan and Kyle Jamieson. 2025. L4Span: Spanning Congestion Signaling over NextG Networks for Interactive Applications. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–25.
 - [36] Jing Wang, Yufan Zheng, Yunzhe Ni, Chenren Xu, Feng Qian, Wangyang Li, Wantong Jiang, Yihua Cheng, Zhuo Cheng, Yuanjie Li, Xiufeng Xie, Yi Sun, and Zhongfeng Wang. 2019. An Active-Passive Measurement Study of TCP Performance over LTE on High-speed Rails. In *The 25th Annual International Conference on Mobile Computing and Networking* (Los Cabos Mexico, 2019-08-05). ACM, 1–16. doi:10.1145/3300061.3300123
 - [37] Ranysha Ware, Adithya Abraham Philip, Nicholas Hungria, Yash Kothari, Justine Sherry, and Srinivasan Seshan. 2024. CCAnalyzer: An Efficient and Nearly-Passive Congestion Control Classifier. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 181–196. doi:10.1145/3651890.3672255
 - [38] Keith Winstein and Hari Balakrishnan. [n. d.]. TCP ex Machina: Computer-Generated Congestion Control. ([n. d.]).
 - [39] Keith Winstein, Anirudh Sivaraman, and Hari Balakrishnan. 2013. Stochastic Forecasts Achieve High Throughput and Low Delay over Cellular Networks. In *Proc. of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (2013).
 - [40] Yaxiong Xie and Kyle Jamieson. 2022. NG-Scope: Fine-Grained Telemetry for NextG Cellular Networks. 6, 1 (2022), 1–26. doi:10.1145/3508032
 - [41] Yaxiong Xie, Fan Yi, and Kyle Jamieson. 2020. PBE-CC: Congestion Control via Endpoint-Centric, Physical-Layer Bandwidth Measurements. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)* (Virtual Event USA, 2020-07-30). ACM, 451–464. doi:10.1145/3387514.3405880
 - [42] Francis Y Yan, Justin Ma, Greg D Hill, Deepti Raghavan, Riad S Wahby, Philip Levis, and Keith Winstein. 2018. Pantheon: the training ground for Internet congestion-control research. In *USENIX ATC* (2018).
 - [43] Wei Ye, Jason Carpenter, Zejun Zhang, Rostand A. K. Fezeu, Feng Qian, and Zhi-Li Zhang. 2023. A Closer Look at Stand-Alone 5G Deployments from the UE Perspective. In *2023 IEEE International Mediterranean Conference on Communications and Networking (MeditCom)* (Dubrovnik, Croatia, 2023-09-04). IEEE, 86–91. doi:10.1109/MeditCom58224.2023.10266621
 - [44] Wei Ye, Xinyue Hu, Steven Sleder, Anlan Zhang, Udhaya Kumar Dayalan, Ahmad Hassan, Rostand A. K. Fezeu, Akshay Jajoo, Myungjin Lee, Eman Ramadan, Feng Qian, and Zhi-Li Zhang. 2024. Dissecting Carrier Aggregation in 5G Networks: Measurement, QoE Implications and Prediction. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 340–357. doi:10.1145/3651890.3672250
 - [45] Wei Ye, Xinyue Hu, Steven Sleder, Anlan Zhang, Udhaya Kumar Dayalan, Ahmad Hassan, Rostand A. K. Fezeu, Akshay Jajoo, Myungjin Lee, Eman Ramadan, Feng Qian, and Zhi-Li Zhang. 2024. Dissecting Carrier Aggregation in 5G Networks: Measurement, QoE Implications and Prediction. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 340–357. doi:10.1145/3651890.3672250
 - [46] Chen-Yu Yen, Soheil Abbasloo, and H. Jonathan Chao. 2023. Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York NY USA, 2023-09-10). ACM, 255–274. doi:10.1145/3603269.3604838
 - [47] Yasir Zaki, Thomas Pötsch, Jay Chen, Lakshminarayanan Subramanian, and Carmelita Görg. 2015. Adaptive Congestion Control for Unpredictable Cellular Networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London United Kingdom, 2015-08-17). ACM, 509–522. doi:10.1145/2785956.2787498